Building Private Apps on Ethereum

A build guide for the Road to Devcon hackathon

DeFi Vertical, Finance and Economics Club, IIT Guwahati

Contents

Before you start

This is a guide to the four privacy tools you can use in this hackathon: Semaphore, Railgun, Privacy Pools and Kohaku. It assumes you can write Solidity and JavaScript, and that you've never touched a zero-knowledge proof.

Here's the thing that surprises people: you won't be writing any cryptography. All four tools ship circuits that are already written and already audited. Your job is to call them correctly and decide what your app does with the answer. Nearly every problem you hit this weekend will be a normal engineering problem wearing unfamiliar words.

Those words are what Part 1 is for. Give it twenty minutes even if some of it looks familiar, because everything later leans on it, and these libraries throw errors that tell you exactly what's wrong once you know what they're naming.

Already have an idea? Read Part 1, then Part 4 if ethers is unfamiliar, then build the project in Part 5.

Still deciding? Part 3 narrows it down in five minutes. Something already broken? Part 13 is a diagnosis table.

1. Vocabulary

Twenty words, in the order they depend on each other. Read them top to bottom, because each one gets used by the ones underneath.

Address. Something like `0x68DD...77c6`. Your MetaMask account is one. It's a name that isn't attached to you yet, which isn't the same as being anonymous.

Transaction. An instruction you sign and broadcast. It permanently records who sent it, which contract it called, what arguments went in, and when.

Gas. What a transaction costs. Somebody has to hold ETH to pay it, and whoever pays gets their address recorded forever. Remember this one. It comes back in Part 9 and ruins things.

Event, or log. Data a contract publishes on purpose so apps can read it. Cheaper than storage, just as public. It's the "Logs" tab on Etherscan.

Hash. A one-way function. Same input, same output, every time, and you can't run it backwards. Two show up here. Keccak256 is what Solidity uses. Poseidon turns up inside proofs because it's much cheaper there, and that's the only reason it's in your code.

Commitment. A hash of a secret that you publish. Safe to publish, since nobody can reverse it. But it also pins you to that one secret, because you can't later claim a different one produced it. A sealed envelope handed to the room.

Merkle tree. Hash things in pairs, hash the results in pairs, keep going until one number covers everything. That number is the root.

Merkle proof. The sibling hashes along the path from your item up to the root. Eight or so numbers for two hundred items, twelve for four thousand. It proves your item is in the tree without handing over the tree.

Circuit. A program written so it can be proved. You won't write one.

Public input. A value the circuit takes that everyone can see. It lands on-chain in the clear. Put something here by accident and you've published it.

Private input. A value the circuit takes that stays on your machine and is never sent anywhere.

Which values go in which pile is your privacy design. That's not a slogan. It's the only decision that really matters, and you make it before writing code.

Zero-knowledge proof. A few hundred bytes proving "I ran the circuit and it came out true", while saying nothing about the private inputs. A second or two to make in a browser. Instant to check.

Nullifier. A number computed from your secret plus some context, published when you act. Same secret and same context give the same number every time, so a contract can spot repeats. But it's a one-way hash, so it can't be traced back to you. That's how you get one-action-per-person and anonymity together, which sounds impossible until you see it.

Scope. The context half of a nullifier. Change the scope and the same person produces a completely different nullifier. Set scope to a poll id and you get one vote per poll, with votes across polls unlinkable.

Anonymity set. How many people you're hidden among. This decides whether you have privacy at all. A perfect proof inside a group of four protects nobody.

Shielded pool. A contract holding everyone's funds together. Money going in and coming out is visible. Movement inside isn't.

Shield and unshield. Putting funds into a pool, and taking them out. Both public.

Note, sometimes called a UTXO. Inside a shielded pool your balance isn't a number. It's a pile of notes, each saying "this much of this token belongs to whoever knows this secret". Spending destroys notes and makes new ones. It works like cash. You don't have 500 rupees, you have a 200 note and three 100s.

Viewing key and spending key. Shielded pools split your key in two. The spending key moves money.

The viewing key only decrypts your own history, so you can give an accountant read access without letting them spend.

Relayer, broadcaster, bundler. Somebody who submits your transaction and pays the gas, so their address is recorded instead of yours. Three names for one idea. Tornado said relayer, Railgun says broadcaster, ERC-4337 says bundler.

A few more turn up inside specific tools.

ERC-4337, user operation, paymaster. A parallel system where you sign a description of what you want done instead of a transaction. A bundler wraps it and pays, and a paymaster can cover the cost. It splits "who authorised this" from "who paid for it".

Association set and ASP. Privacy Pools. An association set is a subset of a pool's deposits that you publicly claim membership in, usually one that leaves out flagged deposits. An Association Set Provider publishes those subsets.

Ragequit. Privacy Pools. Pulling your own deposit back out to the original depositor without needing anyone's approval. An escape hatch that costs you the privacy.

Proof of Innocence, or POI. Railgun's term for proving your funds didn't come from a flagged source.

The two people get wrong

Two of those cause more trouble than all the rest together.

The first is the anonymity set. It's tempting to think privacy comes from the cryptography, and it doesn't. If your group has four members and four actions show up on chain, everybody can work out who did what, and no proof system anywhere saves you. Privacy comes from the crowd you're standing in. The maths only promises you're indistinguishable from the others in your group. No others, no promise.

The second is the gas payer. People build a properly anonymous action, then submit it from their own funded wallet, stamping their address on it forever. The proof stays private and the delivery gives them away. Part 9 deals with this properly, because every project here has the problem whether or not it uses any of these tools.

2. Why privacy is hard on Ethereum

Every node re-runs your transaction and checks it for itself. Do you have the money, is the signature yours, does the contract allow this. They all have to agree or the chain splits, and that independent checking is the whole reason you don't have to trust anybody.

So try encrypting your balance. A node gets your transaction and has to answer "does she have enough money" while staring at ciphertext. It can't. And it can't wave through something it can't verify, because that's the one thing it must never do.

The transparency isn't sloppiness. It falls straight out of how verification has always worked. Which is why privacy here needs a different way to verify, not a clever way to hide.

That's the job a zero-knowledge proof does. Picture a bouncer who needs one fact about you, whether you're over eighteen, and you hand him your Aadhaar card. Now he's got your name, birthday, address, photo and ID number. He needed one bit. A proof closes that gap. You run a program on your own machine using inputs only you have, it spits out a small object, and anyone can check that object and learn exactly one thing.

Three things follow, and you'll use all three.

Proving is slow, verifying is cheap. That's what makes the economics work at all. The expensive half runs on the user's laptop for free, and only the cheap half touches the chain.

You don't write circuits. Every tool here ships audited ones whose proving keys came out of ceremonies with many independent participants. Writing your own is weeks of work, and getting it wrong doesn't throw an error. It quietly lets people prove things that aren't true.

And privacy isn't something you get by importing a library. Bolting a zero-knowledge library onto an app that publishes usernames buys you nothing.

3. Which tool for which problem

Tool	Hides	Doesn't hide	Version
Semaphore	who did something	what was done, amounts	4.14.3, stable
Railgun	balances, amounts, counterparties	that you used it	10.9.0, production
Privacy Pools	which deposit is yours	deposits, withdrawals	1.4.0, production
Kohaku	who submitted the transaction	see Part 10	0.0.1 alpha

The question that sorts it: are you hiding a person, or hiding money?

A person means Semaphore. Voting, feedback, credentials, membership, whistleblowing, claiming something exactly once. The action stays public and the actor doesn't.

Money means Railgun or Privacy Pools. Amounts and balances get concealed.

Hiding who actually sent the transaction is a third thing that sits above both, and it's what relayers and Kohaku are for. No amount of Semaphore or Railgun touches it.

My advice is to start with Semaphore. It's the only one of the four you can integrate in an afternoon, and plenty of ideas that sound like they need money privacy really need identity privacy. "Anonymous grant applications" is Semaphore. "Grant applications where the amounts are secret" is Railgun.

4. Talking to a contract from JavaScript

If you've written Solidity but your JavaScript is the ordinary kind, this is the part that will save you the most time. The cryptography in this guide is genuinely new. This bit isn't new, it's just different enough from `fetch()` to be quietly confusing, and almost every hour lost in a hackathon gets lost here rather than in the proofs.

Your browser doesn't talk to Ethereum, it talks to a node

There's no Ethereum server. There's a network of nodes, and each one exposes a JSON-RPC endpoint: you post a JSON body naming a method, you get JSON back. So "reading a contract" really means posting `eth_call` to some node and getting a hex string, and "sending a transaction" means posting `eth_sendRawTransaction` with a signed blob.

Nobody writes that by hand. Ethers wraps it, and the wrapper has two halves that beginners routinely conflate.

Provider and signer

A **provider** is a read-only connection to a node. It can fetch balances, call view functions, read logs, watch for blocks. It can't change anything, because reading needs no permission and costs nothing.

A **signer** can produce signatures, which means it can authorise state changes. Every transaction has to be signed by whoever's account it comes from, and that's the signer's job.

In a browser, the signer is MetaMask.

```
// MetaMask injects this object into every page. If it's undefined, the
// user hasn't got a wallet installed, and nothing below will work.
window.ethereum

// Wrap it so ethers can use it. This is read-only so far.
const provider = new BrowserProvider(window.ethereum);

// This is the line that pops up MetaMask asking the user to connect.
// Until they approve, you don't know their address and can't sign.
await provider.send("eth_requestAccounts", []);

// Now you can get a signer, which is bound to whichever account they
// picked. Everything that costs gas goes through this.
const signer = await provider.getSigner();
```

The rule of thumb: reading needs a provider, writing needs a signer. If you get an error about a missing signer, you built the contract object with a provider and then called something that changes state.

The ABI is a translation layer

Solidity compiles to bytecode, and bytecode has no idea what a function name is. Calling `vote(proof)` on chain actually means sending a blob of bytes: four bytes identifying the function, then the arguments encoded in a specific layout.

An **ABI** is the JSON description that lets ethers do that encoding. It lists every function, what types go in, what types come out. Hand ethers an address and an ABI, and it hands you back a JavaScript object with matching methods.

```
const contract = new Contract(address, abi, signer);
await contract.vote(proof);   // ethers encodes this using the ABI
```

Two consequences worth knowing before they bite.

If your ABI is out of date, calls fail in ways that don't point at the ABI. You change the contract, forget to recompile, and now ethers encodes the old signature. The node either reverts with nothing useful or, worse, hits a different function that happens to share a selector. Recompile whenever you touch the

contract.

And if you write the ABI by hand, you own that mismatch forever. Import the one your compiler generated instead. That's why the next part imports from `artifacts/` rather than typing out function signatures.

Two kinds of calls, and they behave completely differently

```
// A view function. Free, instant, no wallet popup. Just a read.
const tally = await contract.getTally();

// A state change. Costs gas, pops up MetaMask, takes a block to land.
const tx = await contract.vote(proof);
```

The second one has a trap in it. That `await` resolves as soon as the transaction has been *sent*, not when it's been *mined*. At that moment nothing has happened on chain yet. If you immediately read the tally, you'll get the old one and conclude your contract is broken.

```
const tx = await contract.vote(proof);   // sent
await tx.wait();                         // now it's actually mined
const tally = await contract.getTally(); // now this is current
```

That's why the code in this guide is full of `await (await contract.foo()).wait()`. It looks strange, and it's two different waits doing two different jobs.

BigInt, which will definitely catch you out

Solidity's `uint256` holds numbers far larger than JavaScript's `Number` can represent exactly. So ethers gives you `BigInt` for anything numeric coming off chain, and expects `BigInt` going in.

BigInts are written with an `n` suffix, and JavaScript flatly refuses to mix them with regular numbers.

```
const count = await contract.pollCount();  // this is a BigInt

count + 1        // TypeError: Cannot mix BigInt and other types
count + 1n       // fine
Number(count)    // fine, if you're sure it's small enough to be safe
count.toString() // what you want for display
```

You'll hit `Cannot mix BigInt and other types` at least once. It's almost always a chain value meeting a hardcoded number, or a chain value going into something like `.toFixed()` that only knows about `Number`. Keep values as BigInt through any arithmetic and convert only at the point you render them.

This is also why the next part writes `BigInt(option)` and `scope + 1n` rather than plain numbers.

Errors don't say much by default

A revert comes back as a low-level error, and if your contract uses custom errors (the `error Foo();` style) ethers can decode them only when the ABI knows about them. That's another argument for using the generated ABI: with it, a revert prints `Semaphore__InvalidProof` instead of a hex string.

When you get an unhelpful error, log the whole object rather than `error.message`. The useful part is usually nested in `error.info` or `error.data`.

One environment quirk to carry with you

Some cryptography libraries, snarkjs included, were written for Node and expect a `global` variable that browsers don't have. Bundlers each fix this their own way. In Vite it's one line in the config, which the next part shows. If you move to Next.js or a webpack setup later, you'll need that setup's equivalent, and knowing the *reason* is what lets you find it. The symptom is always the same: `global is not defined`, thrown from somewhere deep inside a library you didn't write.

5. A complete working project

Anonymous voting, from an empty folder to something running in a browser. Copy it, get it working, then turn it into your own project.

If ethers, providers and BigInts aren't second nature yet, read Part 4 first. Everything below assumes it.

There are two halves here, and it's worth being clear about them before any code shows up, because mixing them up is how people get stuck. The contract gets compiled and deployed by Hardhat, which runs in Node on your machine. The identity and the proof get made in the user's browser, because the private key must never leave it. Both halves live in one folder, and Vite serves the browser half.

Setting up

```
mkdir my-project && cd my-project
npm create vite@latest . -- --template vanilla
npm i @semaphore-protocol/core @semaphore-protocol/contracts ethers
npm i -D hardhat dotenv
npx hardhat init          # choose "empty hardhat.config.js"
```

You've now got one folder both tools understand. Hardhat looks in `contracts/` and `scripts/`. Vite serves `index.html`, bundles anything under `src/`, and copies `public/` to the root of the site.

That last bit matters for the next step. The proving keys go in `public/`, and because Vite serves that folder at the site root, the browser finds them at `/zk/...`.

```
mkdir -p public/zk && cd public/zk
curl -O https://snark-artifacts.pse.dev/semaphore/4.13.0/semaphore-12.wasm
curl -O https://snark-artifacts.pse.dev/semaphore/4.13.0/semaphore-12.zkey
cd ../..
```

Call the folder `zk`, not `artifacts`. Hardhat writes compiled contracts into a top-level `artifacts/` folder, and having two different things called artifacts in one project is a good way to confuse yourself at 2am.

That's about 15 MB, and it's the proving key for the circuit. Skip this and the SDK fetches it from a CDN at runtime while your user stares at a blank screen.

Two config files. `hardhat.config.js` for the contract half:

```
require("dotenv").config();
module.exports = {
  solidity: "0.8.23",
  networks: {
    sepolia: {
      url: process.env.RPC_URL ||
           "https://ethereum-sepolia-rpc.publicnode.com",
      chainId: 11155111,
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : []
    }
  }
};
```

And `vite.config.js` for the browser half:

```
export default {
  // snarkjs, which Semaphore uses underneath, expects a browser global
  // that Vite doesn't provide. Without this line, proof generation dies
  // with "global is not defined".
  define: { global: "globalThis" }
};
```

Your deployer key goes in `.env`. Use a throwaway that has never held anything but testnet funds.

```
PRIVATE_KEY=0x...
```

Now, before you write another line, check that `.env` is in your `.gitignore`.

```
grep -q '^\.env$' .gitignore || echo ".env" >> .gitignore
cat .gitignore
```

Vite's template gitignore doesn't include `.env`, so if you skip this you will eventually push a private key to a public repository. Bots scrape GitHub for exactly this and drain the wallet within minutes. It's only a testnet key, so the damage is nothing, but the habit is what matters and you don't want to learn it the expensive way later.

The contract

`contracts/PrivatePoll.sol`. This compiles against the real Semaphore interface.

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.8.23 <0.9.0;

import "@semaphore-protocol/contracts/interfaces/ISemaphore.sol";

contract PrivatePoll {
    ISemaphore public immutable semaphore;
    uint256   public immutable groupId;
    address   public immutable admin;

    string[]  public options;
    uint256[] public tally;
    uint256[] public members;   // so the frontend can rebuild the tree

    mapping(uint256 => bool) public hasJoined;

    event MemberJoined(uint256 indexed commitment, uint256 total);
    event VoteCast(uint256 option, uint256 nullifier);

    constructor(address semaphoreAddress, string[] memory _options) {
        semaphore = ISemaphore(semaphoreAddress);
        // Makes the group with THIS contract as admin, which is what
        // lets addMember below succeed.
        groupId   = semaphore.createGroup();
        admin     = msg.sender;

        for (uint256 i = 0; i < _options.length; i++) {
            options.push(_options[i]);
            tally.push(0);
        }
    }

    // This function is where your project differs from mine.
    function join(uint256 identityCommitment) external {
        require(!hasJoined[identityCommitment], "already joined");
        hasJoined[identityCommitment] = true;

        // Right now anybody can join, so the group proves nothing.
        // Replace this comment with your eligibility rule. See Part 6.

        semaphore.addMember(groupId, identityCommitment);
        members.push(identityCommitment);
        emit MemberJoined(identityCommitment, members.length);
    }

    function vote(ISemaphore.SemaphoreProof calldata proof) external {
        // Both checks are load-bearing. Semaphore proves the proof is
```

```
        // REAL. It has no idea what it's ABOUT. See Part 6.
        require(proof.scope   == groupId,        "wrong scope");
        require(proof.message <  options.length, "bad option");

        // Reverts if this nullifier was used before, or if the proof
        // is forged.
        semaphore.validateProof(groupId, proof);

        tally[proof.message] += 1;
        emit VoteCast(proof.message, proof.nullifier);
    }

    function getMembers() external view returns (uint256[] memory) {
        return members;
    }

    function getOptions() external view returns (string[] memory) {
        return options;
    }

    function getTally() external view returns (uint256[] memory) {
        return tally;
    }
}
```

Deploying it

`scripts/deploy.js`.

```
const hre = require("hardhat");

// Same address on sepolia, base-sepolia, arbitrum-sepolia,
// optimism-sepolia, linea-sepolia and scroll-sepolia. You're not
// deploying Semaphore, you're pointing at the one already there.
const SEMAPHORE = "0x8A1fd199516489B0Fb7153EB5f075cDAC83c693D";

async function main() {
  const Poll = await hre.ethers.getContractFactory("PrivatePoll");
  const poll = await Poll.deploy(SEMAPHORE, ["Yes", "No", "Abstain"]);
  await poll.waitForDeployment();
  console.log("PrivatePoll:", await poll.getAddress());
}

main().catch((e) => { console.error(e); process.exit(1); });
```

```
npx hardhat compile
npx hardhat run scripts/deploy.js --network sepolia
```

Keep the address it prints.

The privacy logic, which is the file to actually read

`src/semaphore.js`. Everything privacy-related lives here, and it's about fifty lines. This runs in the browser.

```
import { Identity, Group, generateProof } from "@semaphore-protocol/core";

// The circuit is compiled separately for each Merkle depth, and every
// depth has its own multi-megabyte proving key. Let the SDK work out
// the depth from group size and the browser silently downloads a NEW
// key every time the group crosses a power of two. So pin it.
// Depth 12 covers 4096 members.
```

```
const DEPTH = 12;

// These paths work because Vite serves public/ at the site root.
const ARTIFACTS = {
  wasm: "/zk/semaphore-12.wasm",
  zkey: "/zk/semaphore-12.zkey"
};

// Deriving the identity from a signature instead of random bytes means
// the user gets the same identity back on another machine just by
// signing again. The trade is that anyone who steals that signature
// owns the identity. Fine for a hackathon. Encrypt and store it
// properly for anything real.
const LOGIN_MESSAGE =
  "Sign in to PrivatePoll. This never leaves your browser.";

export async function createIdentity(signer) {
  const address = await signer.getAddress();
  const cacheKey = `semaphore-identity:${address}`;

  // Without this cache, MetaMask throws up a signing prompt on every
  // single page refresh, which gets old within about a minute of
  // testing. export() gives you a short string, and import() turns it
  // back into the same identity.
  const cached = localStorage.getItem(cacheKey);
  if (cached) return Identity.import(cached);

  const signature = await signer.signMessage(LOGIN_MESSAGE);
  const identity = new Identity(signature);
  localStorage.setItem(cacheKey, identity.export());
  return identity;
  // identity.privateKey stays here, forever, never transmitted.
  // identity.commitment is safe to publish, and goes on-chain.
}

export function buildGroup(commitments) {
  return new Group(commitments.map((c) => BigInt(c)));
}

export async function voteProof({ identity, group, option, scope }) {
  return generateProof(
    identity,        // PRIVATE, never leaves the browser
    group,           // your position in it is PRIVATE
    BigInt(option),  // message, PUBLIC, lands on-chain in the clear
    BigInt(scope),   // scope, PUBLIC, decides the nullifier
    DEPTH,
    ARTIFACTS
  );
  // Comes back with merkleTreeDepth, merkleTreeRoot, nullifier,
  // message, scope and points, which is exactly the struct the
  // contract's vote() wants.
}
```

Caching in `localStorage` is fine for a hackathon and wrong for production, because anything running on your page can read it. The real answer is to encrypt it with something the user provides, or to keep it in memory and let them sign once per session. Keying the cache by address matters too, since switching MetaMask accounts should give you a different identity, not the old one.

Why the browser rebuilds the tree

`buildGroup` looks like busywork. It isn't, and understanding why saves you an hour later.

A Merkle proof is the list of sibling hashes along the path from your leaf up to the root. To produce that list you need the whole tree. But the contract doesn't store the whole tree. It stores the root, which is one number. Storing every node on chain would cost a fortune and buy nothing, because the root is all the contract needs to check a proof against.

So the tree has to get rebuilt somewhere, and your machine is the only place available. The browser reads back every commitment that ever joined, builds an identical tree in memory, and pulls its own siblings out of it.

This is safe. Every commitment is already public, and knowing all of them tells you nothing about who owns which.

It also explains a failure you'll hit. If somebody joins between the moment you build the group and the moment you generate the proof, the contract's root has moved and yours hasn't. Your proof is against a root the contract no longer recognises, and it reverts with `Semaphore__InvalidProof`. Rebuild immediately before proving, every time.

Wiring it up

`src/main.js`. This is the browser half, which is why it talks to `window.ethereum` rather than to Hardhat.

```
import { BrowserProvider, Contract } from "ethers";
import { createIdentity, buildGroup, voteProof } from "./semaphore.js";

// Don't hand-write the ABI. Hardhat generated one when you ran
// `npx hardhat compile`, and it's guaranteed to match your contract.
// Change the contract, recompile, and this updates itself.
import PollArtifact
  from "../artifacts/contracts/PrivatePoll.sol/PrivatePoll.json";

const CONTRACT_ADDRESS = "0x...";   // printed by scripts/deploy.js
const ABI = PollArtifact.abi;

async function connect() {
  if (!window.ethereum) throw new Error("Install MetaMask.");
  const provider = new BrowserProvider(window.ethereum);
  await provider.send("eth_requestAccounts", []);
  return await provider.getSigner();
}

export async function run() {
  const signer   = await connect();
  const contract = new Contract(CONTRACT_ADDRESS, ABI, signer);

  // 1. make the identity. Nothing leaves the browser yet.
  const identity = await createIdentity(signer);

  // 2. publish the commitment. Ordinary transaction.
  await (await contract.join(identity.commitment)).wait();

  // 3. rebuild the tree, right before proving, every time
  const group = buildGroup(await contract.getMembers());
  const scope = await contract.groupId();

  // 4. prove, then vote. This step takes a second or two, because
  //    it's where the proof actually gets made.
  const proof = await voteProof({ identity, group, option: 0, scope });
  await (await contract.vote(proof)).wait();
```

```
  console.log("tally:", await contract.getTally());
}
```

That import only works after `npx hardhat compile` has run at least once, because the file doesn't exist until then. If Vite complains it can't resolve it, that's why.

Hook `run()` to a button in `index.html`, then `npm run dev`.

Making it not look broken

Generating a proof takes a couple of seconds, and it does that work on the main thread. That has a consequence people discover on stage: if you set a loading flag and immediately start proving, the browser never gets a chance to paint the spinner. The tab freezes, the button stays looking clickable, and your demo appears to have crashed.

The fix is to hand control back to the browser for one tick before you start.

```
setStatus("Generating proof, this takes a moment...");

// Let the browser paint that message before we hog the CPU.
await new Promise((resolve) => setTimeout(resolve, 0));

const proof = await voteProof({ identity, group, option, scope });
```

One line, and it's the difference between a demo that looks slow and a demo that looks dead. If you want to do it properly, move proving into a Web Worker so the page stays responsive, but that's a nice-to-have and this is not.

The other thing worth handling is people saying no. Your flow asks the user for a signature and then for a transaction, and they can decline either.

```
try {
  await run();
} catch (e) {
  if (e.code === 4001 || e.code === "ACTION_REJECTED") {
    setStatus("Cancelled.");        // not an error, they changed their mind
    return;
  }
  console.error(e);                 // log the whole object, not e.message
  setStatus(e.shortMessage ?? "Something went wrong.");
}
```

Code `4001` is MetaMask's "user rejected". Treat it as a normal outcome rather than a failure, because a red error banner when somebody simply clicked cancel makes an app feel broken when it's working exactly as designed.

Worth thinking about too: joining the group and voting are two separate transactions. Somebody can approve the first and reject the second, which leaves them in the group having never voted. Your UI should be able to tell that state apart from "hasn't started", or that person can't work out what to do next. The contract already exposes `hasJoined`, so you can just ask it.

Now go break it

Three experiments, in this order.

Vote twice. The second attempt reverts with `Semaphore__YouAreUsingTheSameNullifierTwice`. The contract has no idea who you are. It just recognised the nullifier.

Change the scope to `scope + 1n` and vote again with the same identity. It works, because a different scope makes a different nullifier. One action per scope, and actions in different scopes can't be linked.

Look at your vote on Etherscan. The Logs tab shows the option in the clear with the nullifier beside it. Now call `getMembers()` and try to work out which commitment made that nullifier. You can't. Both numbers are public, sitting on the same screen, and nothing connects them.

Then scroll up to the “From” field. That’s your own address, on a transaction that’s unmistakably a vote. Read Part 9.

6. Semaphore reference

The whole API is six calls

Three in your contract, three in the browser, and they pair up.

In the browser	In the contract	What the pair does
`new Identity()`	`createGroup()`	make an identity, make the set
`identity.commitment`	`addMember(groupId, c)`	register
`generateProof(...)`	`validateProof(groupId, p)`	prove, then check

That's not a summary. There's no seventh call.

What you get

An anonymous membership proof. A nullifier tied to a scope, letting you enforce one action per member per context without knowing who anyone is. A public `uint256` payload riding along with the anonymous action. On-chain verification from a contract that's already deployed, so you write no cryptography. Groups up to depth 32, which is 2 ^ 32 members.

What you don't get

The message isn't hidden. It's a public input, it goes on-chain in the clear, and people get this wrong constantly. If your payload has to be secret, Semaphore is the wrong tool.

Amounts aren't hidden. It's not a payments library.

Persistent reputation doesn't work, because anonymity kills persistence by construction. The way around it is tiered groups. "Has at least five upvotes" becomes a group you prove membership in, rather than a score you carry.

Joining is public, so the member list is public. What's hidden is which member did which thing. Know that, and mention it yourself before somebody else does.

The three decisions that make it your project

Every project here is the Part 5 code with different answers to these. Work them out on paper before opening an editor.

Who gets into the group? This sets your anonymity set, which sets whether you have privacy at all. Open to everybody means the group proves nothing.

```
// token holders
require(token.balanceOf(msg.sender) >= threshold);

// merkle allowlist, cheapest for a fixed list
require(MerkleProof.verify(allowlistProof, allowlistRoot, leaf));

// signed credential from an issuer you control
require(recoverSigner(hash, sig) == issuer);

// one per address
require(!joined[msg.sender]); joined[msg.sender] = true;
```

What does the message carry? One `uint256`, public. An option index, a rating, a bitfield, or the hash of something stored off-chain.

```
// a ranked list, four bits per candidate
uint8 first  = uint8(proof.message & 0xF);
uint8 second = uint8((proof.message >> 4) & 0xF);
```

What does the scope mean? It controls how often each member can act, and which of their actions can be tied together.

Scope value	Effect
`pollId`	one action per member per poll, polls unlinkable
`roundId`	one claim per member per round
a constant	one action per member, ever
`keccak(address(this), pollId)`	same, but won't collide with other apps

That last one matters if people reuse an identity across apps.

The two checks Semaphore won't do for you

`validateProof` guarantees the proof is real. It doesn't guarantee the proof is about what you think it's about.

Drop the scope check and somebody takes a valid proof made for poll 3 and submits it to poll 5. It verifies perfectly, because it really is a valid proof, just not of what you assumed. Semaphore has no idea what a poll is.

Drop the message bound check and somebody votes for option 47 on a three-option poll.

So: anything you assume about the message or the scope, check on-chain. Most application-level bugs in this space are this.

Things that eat an afternoon

Pin the Merkle depth. Covered in Part 5. Skip it and your users watch a spinner for no reason.

Most tutorials online are for version 3. If the code you found talks about `trapdoor` and `nullifier` as two separate identity fields, that's version 3 and it won't run. In version 4 an identity is a keypair.

Rebuild the group right before proving. Covered in Part 5. Roots also expire an hour after they're superseded, because `createGroup` sets `merkleTreeDuration` to one hour.

On Vite, add `define: { global: "globalThis" }`. snarkjs expects it.

7. Railgun

What it actually does

Start with the picture, because the API makes no sense without it.

Railgun is one big contract holding a lot of people's tokens. You send tokens into it, which is called shielding, and that transaction is public. Anybody can see you put 100 USDC into Railgun.

Once the money is inside, things change. Your balance isn't recorded as "this address owns 100 USDC". It's recorded as a note, an encrypted entry saying "100 USDC belongs to whoever knows this secret". Only you can decrypt yours, with a key that lives in your browser.

Paying somebody inside the pool means destroying your note, creating one for them, and proving in zero knowledge that you did the arithmetic honestly without revealing any of the numbers. On chain, all anyone sees is that the pool changed. Not who paid, not who received, not how much.

Taking money out is unshielding, and that's public again.

So the shape is: public in, private in the middle, public out. If one person shields 100 USDC and one person unshields 100 USDC an hour later, that's an easy guess. If a thousand people are shielding and unshielding, it isn't.

What you get

Private transfers where amount, sender and recipient are all hidden. Balance reads with a viewing key. View-only wallets, which are the cleanest compliance demo available to you, because you can show an auditor with full visibility and no ability to spend. Cross-contract calls, letting you unshield, swap on Uniswap, and reshield in one atomic go, so you never publicly appear to hold either token. Broadcasters, who submit on your behalf and take their fee in shielded tokens. And Proof of Innocence.

What you don't get

It doesn't hide that you used Railgun, since shielding and unshielding are public. It isn't a quick integration. And it hides money rather than people, so pair it with Semaphore if you need both.

The pattern behind 182 exports

The package exports 182 things, which looks awful until you notice nearly all of them are three steps of four operations.

```
gasEstimateFor...()   ask how much gas this will cost
generate...Proof()    build the zero-knowledge proof, the slow step
populateProved...()   turn it into transaction data
                      then you sign and send it yourself
```

Shielding skips the middle step, because putting money in is public and there's nothing to hide yet.

The four operations are shield, transfer, unshield, and cross-contract calls. Every one follows the same three steps, so once you've done one you've done all four.

The arguments you'll keep seeing

Railgun's signatures are long, and most of the confusion is just not knowing what the repeated arguments mean.

`txidVersion` is which version of Railgun's transaction format you want. You pass a constant from `shared-models`.

`networkName` or `chain` is which chain you're on.

`railgunWalletID` is the id that `createRailgunWallet` handed you. It isn't an address.

`encryptionKey` is the key encrypting your wallet's local database. Lose it and you lose the local state,

though the mnemonic can rebuild it.

`erc20AmountRecipients` is the list of who gets what. Each entry has a token address, an amount, and a recipient.

`shieldPrivateKey` is a one-time key used to encrypt the note you're creating as you shield. Not your wallet key.

`sendWithPublicWallet` is a boolean: are you paying gas from your own public address, or is a broadcaster paying? If it's true, you've given up on hiding the submitter. See Part 9.

Shielding, in full

Here's a complete shield, with the real signatures. Nothing elided.

```
import {
  startRailgunEngine, loadProvider, createRailgunWallet,
  gasEstimateForShield, populateShield,
  getShieldPrivateKeySignatureMessage
} from "@railgun-community/wallet";
import { NetworkName, TXIDVersion } from "@railgun-community/shared-models";
import { keccak256 } from "ethers";

// Once, at startup
await startRailgunEngine(/* db, artifact store, logging */);
await loadProvider(providerConfig, NetworkName.EthereumSepolia);

const wallet = await createRailgunWallet(
  encryptionKey, mnemonic, creationBlocks
);

// The shield key is derived from a signature over a fixed message that
// the library gives you. It encrypts the note you're about to create,
// and it isn't your wallet key.
const shieldPrivateKey = keccak256(
  await signer.signMessage(getShieldPrivateKeySignatureMessage())
);

// Who gets what. Note the recipient is your "0zk..." address, not your
// Ethereum one, because the money is going into the pool.
const erc20AmountRecipients = [{
  tokenAddress: "0xTokenAddressHere",
  amount: 1000000n,                       // smallest unit, so 1 USDC
  recipientAddress: wallet.railgunAddress  // "0zk..."
}];

// V2_PoseidonMerkle is Railgun's current transaction format. V3 exists
// but isn't live everywhere, so use V2 unless you have a reason not to.
const { gasEstimate } = await gasEstimateForShield(
  TXIDVersion.V2_PoseidonMerkle,
  NetworkName.EthereumSepolia,
  shieldPrivateKey,
  erc20AmountRecipients,
  [],                           // no NFTs
  await signer.getAddress()     // who pays the gas
);

const { transaction } = await populateShield(
  TXIDVersion.V2_PoseidonMerkle,
  NetworkName.EthereumSepolia,
  shieldPrivateKey,
  erc20AmountRecipients,
```

```
  [],
  { evmGasType, gasEstimate, maxFeePerGas, maxPriorityFeePerGas }
);

// Ordinary transaction data. Sign and send it however you normally do.
await signer.sendTransaction(transaction);
```

The `[]` in the middle of both calls is the NFT list. Every Railgun function takes ERC-20s and NFTs as separate arrays, so you pass an empty one when you only care about tokens.

Private transfers

Same three steps with the proof wedged in, but the argument lists are long enough that getting the order wrong is the main way people lose an hour. So here they are as named variables, in the order the functions want them.

I've verified these against the shipped type definitions, not by running them, so the shape is right and you'll still be filling in values.

```
// The pieces, declared once so you can see what goes where.
const txidVersion   = TXIDVersion.V2_PoseidonMerkle;
const networkName   = NetworkName.EthereumSepolia;
const walletID      = wallet.id;
const memoText      = undefined;          // optional note to the recipient
const nfts          = [];                 // ERC-20s only
const erc20AmountRecipients = [{
  tokenAddress:     "0xTokenAddressHere",
  amount:           1000000n,
  recipientAddress: "0zkRecipientAddress"  // their 0zk, not their 0x
}];

// Are you paying gas from your own public address? If true, you've
// given up on hiding the submitter. See Part 9.
const sendWithPublicWallet = false;

// Does the recipient get to learn your 0zk address? This is a product
// decision, not a flag to leave on its default.
const showSenderAddressToRecipient = true;

// 1. estimate
const { gasEstimate } = await gasEstimateForUnprovenTransfer(
  txidVersion, networkName, walletID, encryptionKey,
  memoText, erc20AmountRecipients, nfts,
  originalGasDetails, feeTokenDetails, sendWithPublicWallet
);

// 2. prove. Slow, and it returns nothing, because it caches the proof
//    inside the engine for step 3 to pick up.
await generateTransferProof(
  txidVersion, networkName, walletID, encryptionKey,
  showSenderAddressToRecipient, memoText, erc20AmountRecipients, nfts,
  broadcasterFeeERC20AmountRecipient, sendWithPublicWallet,
  overallBatchMinGasPrice,
  (progress) => console.log(`proving ${progress}%`)
);

// 3. populate. Same list minus encryptionKey and the callback, plus
//    gasDetails on the end.
const { transaction } = await populateProvedTransfer(
  txidVersion, networkName, walletID,
  showSenderAddressToRecipient, memoText, erc20AmountRecipients, nfts,
```

```
    broadcasterFeeERC20AmountRecipient, sendWithPublicWallet,
    overallBatchMinGasPrice, gasDetails
  );

  await signer.sendTransaction(transaction);
```

Step 2 returning `void` catches people out. It looks like it failed. It didn't: the proof is held in the engine, and `populateProvedTransfer` collects it. Which also means the two calls have to agree on every shared argument, or step 3 won't find a proof matching what it's asking for.

Unshielding is the same three steps with `gasEstimateForUnprovenUnshield`, `generateUnshieldProof` and `populateProvedUnshield`. DeFi calls swap in `generateCrossContractCallsProof` and `populateProvedCrossContractCalls`.

Reading a balance needs a sync first, and that sync is the slow part of the whole library. It walks the chain collecting notes and trying to decrypt each one to find out if it's yours. Do it the night before, never during a demo.

```
await refreshBalances(chain, [wallet.id]);
const bal = await balanceForERC20Token(
  txidVersion, wallet.id, chain, tokenAddress, onlySpendable
);
```

A few other functions you might want. `createViewOnlyRailgunWallet` covers the auditor case. `calculateBroadcasterFeeERC20Amount` and `verifyBroadcasterSignature` handle relaying. `refreshReceivePOIsForWallet` and `refreshSpentPOIsForWallet` handle Proof of Innocence. And `getSpendableUTXOsForToken` shows a user their balance as the actual pile of notes it is, which explains the model better than any diagram.

If a signature here doesn't match what you've installed, trust your copy over this page and read the `.d.ts`. Part 11 shows you where.

Ideas

Private payroll. Shield the treasury once, then transfer to each contributor's `0zk` address. All the public sees is a treasury shielding some tokens.

Auditable private treasury. The above, plus a view-only wallet for your auditor. Opaque to everyone, transparent to whoever has a right to look.

Private swap. Cross-contract calls, using a cookbook recipe.

Gotchas

First sync is slow, so never do it live. Proving artifacts download at runtime, though `setArtifactStore` and `setUseNativeArtifacts` let you control that. Proofs expire, so call `validateCachedProvedTransaction` before sending. And `0zk` addresses aren't Ethereum addresses, so check them with `validateRailgunAddress`.

8. Privacy Pools

The idea, before the code

In an ordinary mixer your anonymity set is everybody, criminals included. Honest users get tainted by association and eventually the whole pool gets sanctioned. Privacy and compliance look like opposites.

Privacy Pools pulls them apart. When you withdraw, you prove two things instead of one. First, that your deposit is in the full deposit tree, so it's real money you actually put in. Second, that your deposit is also in a smaller subset you name publicly, the association set, which leaves flagged deposits out.

You get privacy inside the clean subset while showing you're not one of the excluded ones. Somebody with dirty money can't produce that second proof, because their deposit isn't in the set.

The paper is *Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium*, by Buterin, Illum, Nadler, Shaar and Soleimani, 2023. Twenty pages and readable, which is rare. Read it before the SDK and the SDK stops being confusing.

What you get

Deposits in ETH and ERC-20. Withdrawals with an association proof. Ragequit, which pulls your deposit back to the original depositor without needing anyone's approval. Relayer support. Accounts derived from a mnemonic. And enumeration of your own spendable commitments.

What you don't get

It doesn't hide that you used it, since deposits and withdrawals are public. You can't freely invent your own association set in practice, because you pick from what the ASPs publish. And ragequitting sends funds back to the original depositor, which links them, so the privacy goes. That's the trade for having an escape hatch.

Four services

Class	Methods	For
`PrivacyPoolSDK`	`proveWithdrawal`, `proveCommitment`, `verifyCommitment`	top-level entry point
`AccountService`	`account`, `getSpendableCommitments`, `getRagequits`	keys, commitment tracking
`WithdrawalService`	`proveWithdrawal`, `verifyWithdrawal`	the withdrawal proof
`ContractInteractionsService`	`depositETH`, `withdraw`, `relay`, `ragequit`	on-chain calls

The deposit and withdraw flow

The two contract calls have short, verified signatures:

```
depositETH(amount: bigint, precommitment: bigint)
withdraw(withdrawal: Withdrawal, proof: WithdrawalProof, scope: Hash)
```

The proof-building calls take more, and they're in the `.d.ts`. Here's the sequence.

```
import {
  PrivacyPoolSDK, AccountService, ContractInteractionsService,
  generateDepositSecrets, generateMerkleProof
} from "@0xbow/privacy-pools-core-sdk";

// The account comes entirely from a mnemonic. Two master keys fall out
// of it, one for nullifiers and one for secrets. Back the mnemonic up.
// It IS the account.
const account = new AccountService(dataService, { mnemonic });
```

```
// Secrets for this particular deposit
const secrets = generateDepositSecrets(account, scope, index);

// Deposit. Public transaction.
const contracts = new ContractInteractionsService(config);
await contracts.depositETH(amount, secrets.precommitment);
```

Then you wait. Not because the code needs it, but because your anonymity set is whoever else deposited, and withdrawing thirty seconds after depositing is trivially linkable no matter how good the proof is.

The withdrawal itself runs in four steps: list what you can spend, build a Merkle proof against the ASP's set, prove the withdrawal, then submit it or hand it to a relayer.

```
// What can I spend?
const commitments = await account.getSpendableCommitments({
  includeEmptyNodes: false     // default is true, and you rarely want it
});

// Prove membership of the association set the ASP published
const merkleProof = generateMerkleProof(associationSetLeaves, myCommitment);

// Build the withdrawal proof
const sdk = new PrivacyPoolSDK(circuits);
const withdrawalProof = await sdk.proveWithdrawal(commitment, input);

// Submit it yourself, or hand it to a relayer so your address isn't
// the one recorded
await contracts.withdraw(withdrawalProof);
// or: await contracts.relay(withdrawalProof, relayerDetails);
```

`ragequit` is the escape hatch. It returns your deposit to the original depositor without any ASP approving you, and in doing so it links the deposit to the depositor. Handy to have. Don't demo it as though it were a private withdrawal.

One argument worth knowing about before it surprises you: `getSpendableCommitments` includes empty, migrated and ragequit nodes by default. Pass `includeEmptyNodes: false` unless you specifically want them, or you'll be looking at a list that doesn't match what you can actually spend.

Ideas

A clean-funds grant program. Disburse from the pool and prove every payout came from an association set that excludes flagged deposits. Privacy for recipients, provenance for auditors.

An association set explorer. Show which sets a deposit qualifies for, and how the anonymity set shrinks as the set gets stricter. Making that trade-off visible is a good demo and needs no new cryptography.

Your own ASP. Publish a set for a specific community. Real missing infrastructure, and small enough to actually build.

9. The leak every project has

What it is

You build a properly anonymous action. A user makes a proof in their browser, and nothing about their identity is in it. Then they submit it from their own funded wallet, paying gas from an address linkable to their name.

The anonymity is gone at the last step. The proof was fine. The delivery gave them away.

This isn't a flaw in Semaphore or in any of the other three. It sits one layer below all of them, because somebody has to pay for the transaction and Ethereum records who. Your project has this. So does everybody else's.

The cheap fix

Don't have the user submit their own proof.

A proof stands on its own. It doesn't need to arrive from any particular address to be valid, which means you can hand it to somebody else and let them submit and pay. Their address gets recorded. You aren't trusting them with anything, because the proof already proves what it proves and they can't alter it.

The simplest version, and it's perfectly reasonable here, is a small server you run yourself. The browser posts the proof to your endpoint, the endpoint submits the transaction from a wallet you funded, and every vote in your demo arrives from the same address. An hour of work, and it turns "we know about this problem" into "we fixed this problem".

The honest caveat is that your server now knows which IP sent which proof, so you've moved the trust rather than removed it. Say so. A project that explains where its remaining trust sits reads much better than one claiming there is none.

Production systems use networks of independent relayers, so no single operator sees everything. Railgun runs one and calls the participants broadcasters.

Building one

Two files. Here's the browser side, replacing the direct `contract.vote(proof)` call from Part 5.

```
// Before: the user submits, and their address is on it forever.
// await (await contract.vote(proof)).wait();

// After: hand the proof to your own endpoint and let it submit.
const res = await fetch("http://localhost:3001/relay", {
  method:  "POST",
  headers: { "Content-Type": "application/json" },
  // BigInts don't survive JSON.stringify, so stringify them yourself.
  body: JSON.stringify({
    merkleTreeDepth: Number(proof.merkleTreeDepth),
    merkleTreeRoot:  proof.merkleTreeRoot.toString(),
    nullifier:       proof.nullifier.toString(),
    message:         proof.message.toString(),
    scope:           proof.scope.toString(),
    points:          proof.points.map(String)
  })
});

const { txHash, error } = await res.json();
if (error) throw new Error(error);
```

Notice the user's wallet is never touched here. They signed once, long ago, to create their identity. Casting the vote needs no signature at all, because the proof is the authorisation.

And the server:

```
// relay.js   (run it with: node relay.js)
// npm i express cors ethers @semaphore-protocol/core
import express from "express";
import cors from "cors";
import { Wallet, JsonRpcProvider, Contract } from "ethers";
import { verifyProof } from "@semaphore-protocol/core";
import PollArtifact
  from "./artifacts/contracts/PrivatePoll.sol/PrivatePoll.json" with { type: "json" };

const provider = new JsonRpcProvider(process.env.RPC_URL);
const relayer  = new Wallet(process.env.RELAYER_KEY, provider);
const contract = new Contract(process.env.CONTRACT, PollArtifact.abi, relayer);

const app = express();
app.use(cors(), express.json());

app.post("/relay", async (req, res) => {
  try {
    const proof = req.body;

    // Check the proof here, before spending any gas on it. A bad proof
    // would revert on chain anyway, but you'd have paid for the revert.
    if (!(await verifyProof(proof))) {
      return res.status(400).json({ error: "invalid proof" });
    }

    const tx = await contract.vote(proof);
    await tx.wait();
    res.json({ txHash: tx.hash });
  } catch (e) {
    // Reverts land here. The common one is a repeated nullifier, which
    // means somebody already voted with that identity.
    res.status(400).json({ error: e.shortMessage ?? e.message });
  }
});

app.listen(3001, () => console.log("relayer on :3001"));
```

That's the whole thing. The relayer holds a funded key, checks the proof, and submits. Every vote in your demo now arrives from the same address, and that address tells an observer nothing about who voted.

The hole you just opened

An endpoint that pays gas on request is a wallet anybody on the internet can spend. Before you leave it running, three things:

Verify before submitting. The `verifyProof` call above is the important line. Without it, anyone can POST junk and make you pay for a revert every time.

Pin down what it will submit. The handler above calls exactly one function on exactly one contract, with a proof it has already checked. Don't be tempted to accept a target address or arbitrary calldata from the request body, because that turns your relayer into a general-purpose wallet for strangers.

Rate limit it. `express-rate-limit` in one line is enough for a weekend. And fund the relayer with the small amount of testnet ETH you actually need, so the worst case is that it runs dry rather than that something valuable disappears.

Say all of this in your write-up. The relayer is a real improvement and it is also a trusted party who can see which IP sent which proof, refuse to submit, and reorder submissions. Naming that is the difference between a project that understands its own trust model and one that doesn't.

The general version of the idea

The trouble with per-protocol relayer networks is that every protocol built its own, each with separate operators, fee logic and trust assumptions, and each small enough to shut down.

ERC-4337 generalises it. You sign a description of what you want done, called a user operation, rather than a transaction. Independent bundlers pick it up, wrap it in a real transaction and pay for it, and the address on chain is theirs. It's ordinary account abstraction plumbing that plenty of wallets already use, not a privacy-specific trick, and that's what makes it likely to stick around.

10. Kohaku

Kohaku is the Ethereum Foundation's attempt to make privacy something a wallet does, rather than something you visit a separate website for. It's early and it's incomplete, and understanding it properly is a genuinely useful way to spend this weekend.

The architecture

Two ideas.

The first is that every shielded pool should look identical to a wallet. Railgun, Privacy Pools and Tornado all do fundamentally the same three things, so Kohaku defines one interface and each pool becomes a plugin behind it. A wallet integrates once instead of three times, and a user picks a pool the way they pick a network.

```
type PluginInstance = {
  instanceId:  () => Promise<string>
  balance:     (assets) => Promise<AssetAmount[]>
  prepareShield:   (asset, to) => Promise<PublicOperation>
  prepareTransfer: (asset, to) => Promise<PrivateOperation>
  prepareUnshield: (asset, to) => Promise<PrivateOperation>
  broadcastPrivateOperation: (op) => Promise<void>
}
```

Shield, transfer, unshield. The same three verbs no matter what's underneath.

The second idea is the delivery problem from Part 9. Rather than every protocol running its own relayers, Kohaku routes privacy transactions through the shared ERC-4337 mempool, so bundlers that already exist do the submitting and paying.

Plugins assume nothing about their environment. You hand them a Host:

```
type Host = {
  network:  Network            // how to make requests
  storage:  Storage            // where synced state gets kept
  keystore: Keystore           // key material
  provider: EthereumProvider   // RPC access
}
```

That's a good design. It's also mostly a design rather than a working library right now, which is the interesting part.

What works and what doesn't

I installed the packages and tried to run them. Here's where it stands.

`@kohaku-eth/provider` works. It's a small abstraction that sits on top of whichever library you already use for RPC, exporting an `EthereumProvider` type and a `createTx(address, payload, value)` helper. It supports Ethers and Viem, which are the two mainstream JavaScript libraries for talking to Ethereum and do roughly the same job, plus Helios and Colibri, which are light clients that verify what a node tells them rather than trusting it. Install it and use it today.

`@kohaku-eth/plugins` won't import under Node ESM. Its `dist/index.js` re-exports `./base` with no file extension, and Node ESM requires explicit extensions. Bundlers tolerate it, which is presumably why this passes their CI.

`@kohaku-eth/railgun` won't import either, because it uses a directory import, `dist/pkg`, which Node ESM doesn't support.

Neither loads under CommonJS, since there's no CommonJS export map.

Both do build under Vite, but only after manually installing `viem`, which isn't declared as a dependency. And I'd expect the browser bundle to fail at runtime anyway, because the Railgun WASM loader reaches for `node:fs`, `node:path` and `node:url`.

The README agrees: not production ready, unaudited, breaking changes expected between versions.

What to build

Don't build a product whose demo depends on the broken parts. Build around what exists.

Explain the architecture properly. Nobody has written a clear account of what Kohaku is trying to do and why the plugin abstraction matters. A good walkthrough, as a document or a small site, with the plugin lifecycle drawn out and the 4337 routing explained, is a real contribution and something you can actually finish in a weekend. Do it well and it becomes the thing people link to.

Map the gaps. Go further than I did. Try the browser path, work through each package, and record exactly what breaks, with reproduction steps, package versions and the errors themselves. An honest gap analysis is worth more than a half-working demo, and filing it as an issue on `github.com/ethereum/kohaku` turns it from a hackathon artifact into a contribution to the project.

Build on the parts that work. `@kohaku-eth/provider` is usable, so build something small on it and show the abstraction earning its keep. Or take `PluginInstance` as a specification and implement it yourself against something simple. A working plugin that conforms to Kohaku's own interface demonstrates the design is sound even where the implementation isn't, and that's a more interesting result than getting their Railgun plugin to import.

```
pnpm add @kohaku-eth/provider
# and, if you want to attempt the rest:
pnpm add @kohaku-eth/plugins @kohaku-eth/railgun viem
```

Versions are `provider@0.1.0-alpha.8`, `plugins@0.0.1-alpha.11`, `railgun@0.0.1-alpha.28` and `privacy-pools@0.0.1`. The `plugins` package ships `MemoryStorage` and `MnemonicKeystore`, so use those rather than writing a Host from scratch, and set `logLevel: "Debug"` straight away, because it defaults to off and the logs are the only documentation there is.

11. Working from the official documentation

This part will matter most over the next two days, because you'll hit things this guide doesn't cover. Documentation for tools this young is patchy, sometimes wrong, and often describes a version you aren't running.

The type definitions are the real reference

Every package you install ships its own API description, and unlike a website it's guaranteed to match the code you're running.

```
ls node_modules/@semaphore-protocol/core/dist/types/
find node_modules/@railgun-community/wallet -name "*.d.ts" | head
```

Those `.d.ts` files list every function, its parameters and its return type. When documentation and types disagree, the types win. Every function list in Parts 6 and 7 came from there rather than a website.

Two commands go with this.

```
node -e "console.log(Object.keys(require('@railgun-community/wallet')))"
npm view @semaphore-protocol/core versions --json | tail -5
```

The first shows what a package really exports. The second shows whether you're behind.

Tests and examples beat prose

Nearly all these repositories have tests, and tests are usage examples that are verified to work. When a documentation page hands you a fragment with three things elided, go and find the test that calls the same function and read that instead. Same for `examples/` folders, which is the only real documentation Kohaku has.

Where to look, tool by tool

Semaphore. Start at `docs.semaphore.pse.dev`. The docs are good, but check the URL doesn't have `/V2/` or `/V3/` in it, because search engines love serving those and the version 3 API is different enough to cost you an evening. The spec is at `github.com/privacy-ethereum/zkspecs` and it's short. The monorepo is `github.com/semaphore-protocol/semaphore`, where `packages/` maps one to one onto what you installed.

Railgun. The wallet package's exports are the API reference, and reading them grouped by prefix beats any page. Source is at `github.com/Railgun-Community`. Check the cookbook before writing DeFi integration code yourself.

Privacy Pools. Paper first, then the `@0xbow/privacy-pools-core-sdk` types. The four service classes in Part 8 are the whole surface.

Kohaku. No documentation exists. Read `dist/**/*.d.ts` and the `examples/` folder in `@kohaku-eth/plugins`. The doc comments inside the type files are better than you'd expect.

Read CHANGELOG.md before upgrading

All four move fast, and Kohaku's alpha versions ship breaking changes between patch numbers. If something worked yesterday and doesn't today, check whether a version moved before blaming your own code.

A workflow that saves hours

Get the smallest possible thing working before building on it. One identity created and printed. One deposit landing on chain. One balance read. Commit that, then extend. Debugging one new call against something you know works is a completely different job from debugging a whole feature at once, and it's the difference between finishing and not.

12. Combining tools

```
DELIVERY     Kohaku, ERC-4337, relayers, broadcasters
             who submits and pays
.....................................................
PROTOCOLS    Semaphore        |  Railgun, Privacy Pools
             hides WHO        |  hide WHAT and HOW MUCH
.....................................................
PRIMITIVES   commitment, merkle tree, nullifier
```

Semaphore plus a relayer is the best combination available to you this weekend, and the cheapest. Semaphore handles anonymity, any relayer handles delivery, and together they close the Part 9 leak without touching alpha software.

Semaphore plus Railgun gets both halves. Semaphore decides who's allowed to act, Railgun moves the money privately.

Privacy Pools plus your own ASP lets the pool handle the money while you build association-set logic for a specific community.

What ties all four together is that they're the same three primitives arranged differently. Semaphore commits to an identity, a shielded pool commits to a note. Both put commitments in a Merkle tree. Both use nullifiers to stop double use, one blocking a second vote and the other a second spend. If you built the Part 5 project you already understand the shape of a mixer, which is worth knowing before deciding something is beyond you.

13. Diagnosing problems

Work out which layer broke first

Before searching for anything, decide which of these four it is. They have completely different causes, and mixing them up means searching for the wrong thing.

Layer	What it looks like
Install	the import or require fails, module not found
Call	the JS function throws before any transaction
Proof	`generateProof` hangs, or the proof is rejected
Contract	the transaction reverts

Semaphore errors

`Semaphore__YouAreUsingTheSameNullifierTwice`

Working correctly. This identity already acted in this scope. If that isn't what you expected, your scope is probably wrong.

`Semaphore__InvalidProof`

Nearly always a stale root. Somebody joined after you built the group. Reload members, rebuild the group, generate the proof again.

`Semaphore__MerkleTreeRootIsExpired`

The root you proved against is older than `merkleTreeDuration`, which defaults to one hour. Rebuild from current members.

`Semaphore__MerkleTreeDepthIsNotSupported`

Your depth is outside 1 to 32. Check what you pinned.

`Semaphore__GroupHasNoMembers`

Nobody has joined yet. Join first.

`Failed to fetch ...semaphore-N.wasm`

The artifacts aren't there, or your depth changed and it's asking for a file you never downloaded. Pin the depth, download the artifacts.

Proof generation hangs forever. Out of memory, almost always on mobile. Desktop only, and close some tabs.

Everything else

`insufficient funds for intrinsic transaction cost`

The account doing the signing has no ETH. Worth checking that your MetaMask account is the one you funded, since it usually isn't the deploy key.

`Cannot find module '.../dist/base'`

JavaScript has two module systems. The old one, CommonJS, uses `require()` and guesses at file extensions. The newer one, ES modules, uses `import` and demands you spell the extension out. Node enforces the strict rules, and bundlers like Vite don't, so a package can be broken in Node and fine in the browser. If it's your own code, add the `.js`. If it's a dependency, the package is at fault. See Part 10.

A `.env` change had no effect. Vite doesn't hot-reload env files. Restart the dev server.

Nonce errors after restarting a local node. MetaMask cached the old chain state. Settings, Advanced, Clear activity tab data.

```
global is not defined
```

From snarkjs. Add `define: { global: "globalThis" }` to your Vite config.

Read the whole error

These libraries throw unusually specific errors. `Semaphore__MerkleTreeRootIsExpired` tells you exactly what's wrong once you know the vocabulary from Part 1. That's most of the reason Part 1 exists.

14. Scoping your project

Answer the three decisions from Part 6 on paper before opening an editor. Then build the smallest thing that performs one anonymous action end to end, and get it deployed and working before adding anything else.

Private identity and credentials. Semaphore. The whole project is the eligibility rule inside `join()`. Users then prove they hold a valid credential without revealing which one.

Anonymous governance. Semaphore. Change what the message and the scope mean. Ranked choice packs a ranking into one `uint256`. Weighted voting needs one group per tier, because you can't put a weight in the proof without leaking it. Quadratic voting needs per-voter state, which breaks anonymity, so if you go after it, present the difficulty honestly rather than hiding a broken version.

Confidential payments. Either Semaphore deciding who may claim, with fixed amounts, or integrate an existing pool. Don't write your own. Fixed amounts matter, because variable ones fingerprint people.

Privacy-first consumer apps. Usually Semaphore, and the bar is product rather than cryptography. Not a message board with zero-knowledge bolted on, but something only possible because of the anonymity.

Ideas, if you're stuck

What you want to build	Tool	Features involved
Anonymous voting	Semaphore	group, scope per poll
Course or faculty feedback	Semaphore	group of enrolled students
Whistleblowing platform	Semaphore	message as report hash
Sybil-resistant airdrop	Semaphore	nullifier, scope per round
Credential without disclosure	Semaphore	gated `join()`
Anonymous reputation	Semaphore	tiered groups
DAO voting with secret positions	Semaphore	one group per weight tier
Private payroll	Railgun	shield, private transfer
Auditable private treasury	Railgun	view-only wallet
Swap without appearing to hold	Railgun	cross-contract calls
Donations with clean provenance	Privacy Pools	association sets
Grants proving funds are untainted	Privacy Pools	withdrawal proof
Privacy and compliance trade-off	Privacy Pools	association set explorer
Kohaku architecture explainer	Kohaku	plugin interface, 4337 routing

Two things that improve any of them

Grow your anonymity set. Go round the other teams and get twenty people to join your group. Ten minutes of work, and worth more than a feature, because a demo with six members visibly doesn't work.

And know what your project still leaks. Every one of them leaks something: the join transaction, the timing, the gas payer, your RPC provider seeing your IP. Naming it yourself reads as competence. Getting caught reads as not understanding your own project.